КРИТЕРИИ ОЦЕНКИ ПРИЗНАКОВ

Как в EH/Unwind Analyzer назначаются веса, пороги, confidence, корреляции и итоговый verdict

Основа документа: `docs/evidence_matrix.md`, `src/heuristics.c`, `src/report.c`, `docs/threshold_calibration.md`.

Назначение документа

Этот PDF нужен для защиты и объясняет не только список 27 findings, но и критерии их оценки. Главная идея: проект не заявляет математически оптимальную модель и не выдает вероятность вредоносности. Он использует воспроизводимую инженерную шкалу подозрительности, где каждый признак имеет понятную техническую причину, вес, severity, влияние на confidence и роль в итоговом verdict.

Важно разделять четыре вещи: finding - конкретный сигнал; score - сила набранных технических аномалий; confidence - устойчивость интерпретации; final verdict - итоговая категория отчета. Один finding не равен verdict. Итог строится по совокупности сигналов, их семейств, severity, корреляции и динамических данных.

1. Базовые понятия

Термин	Смысл в проекте
Finding	Структурированный сигнал с code, family, severity и message. Пример: unwind_rva_oob или handler_writable_section.
Severity	Качественная сила одного признака: medium или high. High ставится там, где нарушается базовая целостность PE/EH/unwind или возникает трудно объяснимый layout.
Score	Число 0..100. Это инженерная шкала suspiciousness, а не процент вредоносности. Score суммирует веса findings и насыщается сверху.
Suspicion	Нормализованное представление score: score / 100. Оно удобно для JSON, но тоже не является вероятностью.
Confidence	Уверенность в интерпретации статического verdict. Она растет от разнообразия и силы признаков, но падает при одиночных, малых или односемейных сигналах.
Mitigation	Ослабляющее правило. Оно не удаляет finding и не уменьшает score, но снижает confidence, если есть риск переинтерпретации.
Case strength	Итоговая сила случая в assessment: no_case, weak_case, medium_case, strong_unconfirmed_case, medium_correlated_case, strong_correlated_case.

2. Почему веса не одинаковые

Веса отражают не “опасность вируса”, а техническую силу нарушения. Чем ближе признак к нарушению базовой координатной системы PE, function table или UNWIND_INFO, тем выше вес. Чем больше у признака легитимных объяснений, тем ниже вес или тем больше условий для его включения.

Вес	Класс	Почему так
+25	Максимально сильные статические нарушения	Используется для грубого повреждения структуры: выход за границы файла/образа, повреждение Exception Directory, перекрытие RUNTIME_FUNCTION, массовый parse fail. Такой сигнал сложно объяснить обычной компиляцией.
+20	Сильные, но чуть менее абсолютные сигналы	Например, virtual OOB, сильный reuse, executable unwind metadata. Это серьезно, но иногда требует контекста, поэтому не все такие признаки получают максимум.
+15	Заметные структурные или placement-сигналы	Они важны для triage, но могут быть вторичными симптомами или зависеть от toolchain/runtime. Пример: raw overlap, rf_unsorted, handler_nonexec_section, handler_dominance.
+10	Умеренные признаки и небольшие combo-бонусы	Используются там, где сигнал важен, но сам по себе не должен резко доминировать verdict: no_runtime_functions, duplicate_begin, writable handler, moderate reuse.
+8	Мягкие или подтверждающие признаки	Используются для слабых placement/combination-сигналов. Они повышают общую картину, но не превращают одиночную странность в сильный verdict.

Именно поэтому проект не делает все признаки равными. Например, выход Exception Directory за границы образа сильнее, чем одиночное необычное имя секции. A moderate reuse включается только при подтверждении другим статическим сигналом, потому что повторное использование metadata может иметь легитимные runtime-причины.

3. Evidence families

Family	Что оценивает
`table_integrity`	Целостность PE/exception/function-table координат: секции, Exception Directory, RUNTIME_FUNCTION, Begin/End/UnwindInfoAddress.
`unwind_integrity`	Корректность чтения и структуры UNWIND_INFO: parse fail, anomalies, связь placement с повреждением unwind metadata.
`metadata_placement`	Где физически лежат unwind metadata и handler target: executable, writable, non-executable, unusual sections.
`unwind_reuse`	Насколько много функций разделяют один UnwindInfoRVA. Важен не сам факт reuse, а масштаб и подтверждение другими сигналами.
`handler_dominance`	Насколько один handler target доминирует над большим числом функций. Само по себе может быть runtime-нормой, поэтому требует контекста.

Количество разных семейств важно для confidence: если срабатывания пришли из нескольких независимых областей, интерпретация устойчивее. Если все сигналы из одного семейства и нет high finding, применяется ослабление single_family_only.

4. Матрица 27 признаков: вес и причина

Ниже не просто список findings, а объяснение, почему конкретный признак имеет такой вес. Формулировка “почему” пригодна для устного ответа на защите.

Finding code	Severity	Вес	Почему именно так
`pe_section_raw_oob`	high	+25	Raw-диапазон секции выходит за пределы файла. Это базовое нарушение структуры PE на уровне физического файла, поэтому вес максимальный.
`pe_section_virtual_oob`	high	+20	Виртуальный диапазон секции выходит за SizeOfImage/image layout. Это сильная ошибка координат RVA, но обычно требует сверки с остальными заголовками.
`pe_section_raw_overlap`	medium	+15	Raw-диапазоны секций пересекаются. Это делает принадлежность байтов неоднозначной, но иногда может быть результатом упаковки или нестандартной сборки.
`pe_section_virtual_overlap`	high	+20	Виртуальные диапазоны секций пересекаются. Один RVA может относиться к нескольким секциям, что ломает карту анализа.
`pe_section_misaligned`	medium	+8	Нарушено выравнивание секций. Это слабее OOB/overlap, потому что нестандартное выравнивание возможно в специальных образах.
`pe_exception_directory_oob`	high	+25	Exception Directory указывает за пределы корректного PE-образа. Для EH/unwind анализа это критично, так как оттуда начинается поиск .pdata.
`no_runtime_functions`	medium	+10	Нет записей RUNTIME_FUNCTION. Сигнал слабее, потому что leaf-only или очень маленький образ может легитимно не иметь записей.
`invalid_ranges`	medium/high	+10 / +20	BeginAddress/EndAddress образуют плохой диапазон. До 1% это может быть единичный дефект, выше 1% - широкий сбой карты функций.
`rf_oob`	medium/high	+10 / +25	Диапазон функции выходит за SizeOfImage. Единичный случай получает умеренный вес, массовый - максимальный, потому что ломает связь function table с PE-образом.
`unwind_rva_oob`	medium/high	+10 / +25	UnwindInfoAddress ведет в недопустимый RVA. Это напрямую ломает переход от RUNTIME_FUNCTION к UNWIND_INFO.
`rf_unsorted`	medium	+15	Function table не отсортирована. Это влияет на поиск функции по RIP/RVA, но само по себе может быть менее разрушительно, чем OOB.
`rf_overlaps`	high	+25	Диапазоны функций перекрываются. Один адрес может попадать сразу в несколько RUNTIME_FUNCTION, поэтому карта функций становится противоречивой.
`duplicate_begin`	medium	+10	Несколько функций имеют одинаковый BeginAddress. Это неоднозначность, но обычно менее сильная, чем полное перекрытие диапазонов.
`unwind_parse_fail`	medium/high	+10 / +25	UNWIND_INFO не разбирается. Единичный сбой может быть локальным, массовый parse fail означает серьезное повреждение xdata.
`unwind_anomalies`	medium/high	+10 / +20	Поля UNWIND_INFO выглядят странно: version, opcode, chain, frame register. Порог 5% отделяет локальную аномалию от системного паттерна.
`unwind_reuse_moderate`	medium	+10	Один UnwindInfoRVA используется заметной долей функций, но finding появляется только при подтверждении другим сигналом.
`unwind_reuse_strong`	high	+20	Сильная концентрация функций на одном UnwindInfoRVA. Это может выглядеть как искусственная или dispatcher-style metadata.
`handler_dominance`	medium	+15	Один handler target доминирует над многими функциями. Вес не high, потому что централизованные runtime handlers иногда бывают легитимными.
`unwind_in_exec_section`	high	+20	UNWIND_INFO лежит в исполняемой секции. Metadata обычно ожидается в неисполняемых областях, поэтому это сильный placement-сигнал.
`unwind_unusual_section`	medium/high	+8 / +15	UNWIND_INFO лежит в нетипичной секции. До 5% это мягкий сигнал, выше 5% - уже pattern размещения.
`unwind_writable_section`	medium/high	+10 / +18	UNWIND_INFO находится в writable-секции. Это сильнее unusual placement, потому что рецепт unwind потенциально изменяем.
`handler_nonexec_section`	high	+15	Handler target указывает в неисполняемую или неизвестную секцию. Severity high, но вес 15, потому что нужно проверить RVA и section mapping.
`handler_writable_section`	medium	+10	Handler target связан с writable-секцией. Это важно, но одиночный сигнал лучше подтверждать другими placement/integrity findings.
`placement_exec_nonexec_handler_combo`	high	+10	Комбинация: unwind metadata в executable section и handler в non-exec/unknown. Вес как бонус, потому что базовые признаки уже посчитаны.
`placement_integrity_combo`	medium	+8	Placement-проблема подтверждается parse/integrity-проблемой. Это усиливает картину, но остается дополнительным весом.
`reuse_dominance_combo`	medium	+8	Reuse metadata сочетается с handler dominance. Это похоже на централизованную схему, но требует ручной проверки.
`writable_metadata_handler_combo`	high	+10	Writable одновременно metadata и handler target. Это сильная связка, но вес бонусный, чтобы не дублировать уже начисленные базовые признаки.

5. Пороги и почему они такие

Пороги в проекте являются engineering defaults. Они нужны, чтобы отделить одиночную локальную странность от широкого паттерна. Документ threshold_calibration прямо фиксирует: текущие defaults не являются универсальной статистической калибровкой и должны уточняться на расширенном corpus.

Порог	Где применяется	Смысл
> 1% записей	invalid_ranges, rf_oob, unwind_rva_oob, unwind_parse_fail	Если повреждена одна запись, это может быть локальный дефект. Если больше 1% записей, это уже не выглядит как единичная ошибка и повышается severity/weight.
> 5% записей	unwind_anomalies, unwind_unusual_section, unwind_writable_section	Порог выше, потому что placement и некоторые anomalies могут зависеть от компилятора, линкера, packer-а или runtime. Широкий pattern важнее одиночного случая.
n > 200 и reuse > 10%	unwind_reuse_moderate	На малых таблицах проценты легко искажаются. Moderate reuse не считается сам по себе: нужен другой статический сигнал.
n > 200 и reuse > 30%	unwind_reuse_strong	30% означает сильную концентрацию функций на одном UnwindInfoRVA. Это уже заметная структура, а не случайное совпадение.
handler_count > 50 и dominance > 80%	handler_dominance	Один handler на малой выборке не должен считаться подозрительным. Нужны и достаточное число handler-записей, и очень высокая доля одного target.
score >= 20	manual review	Даже если final verdict еще не высокий, сумма сигналов уже достаточна, чтобы аналитик открыл PE viewer, Ghidra или debugger.

> Если комиссия спросит “почему не другие числа”, правильный ответ: это консервативные инженерные defaults, зафиксированные в evidence matrix и вынесенные в threshold calibration workflow. Они выбраны так, чтобы не переоценивать одиночные странности и требовать ручной проверки там, где сигнал зависит от контекста.

6. Confidence model

Confidence показывает не вероятность вредоносности, а устойчивость интерпретации. Он растет, когда признаки сильные и разнообразные, и падает, когда срабатывание одиночное, выборка маленькая или вся картина держится на одном семействе.

Правило	Почему
`base = 0.15`	Начальная осторожная уверенность при наличии анализа.
`score == 0 -> 0.20`	Нулевая подозрительность тоже имеет базовую уверенность, но она не абсолютна.
`per family +0.11`	Разные evidence families дают независимую поддержку интерпретации.
`per high/critical +0.08`	Сильные findings повышают confidence больше, чем medium.
`per medium +0.04`	Средние findings тоже учитываются, но мягче.
`hard table issue +0.08`	Invalid ranges, OOB, overlaps говорят о проблемах карты функций.
`hard unwind issue +0.08`	Parse fail или unwind anomalies говорят о проблемах рецепта unwind.
`clamp 0.05..0.90`	Статический анализ не должен выдавать 0% или 100% уверенность.

Ослабляющие правила

Mitigation	Логика
`single_signal_only`	Ровно один non-strong finding. Снижение -0.12: один слабый сигнал нельзя превращать в уверенный вывод.
`small_sample_size`	Меньше 32 функций, есть findings, нет strong. Снижение -0.06: проценты на маленькой .pdata легко выглядят громче реальности.
`placement_only_signal`	Есть только unusual placement без exec/non-exec/parse/range damage. Снижение -0.10: нестандартная секция еще не значит повреждение.
`single_family_only`	Нет high finding и evidence из одного family. Снижение -0.08: нужна cross-layer corroboration.

7. Корреляция и combo findings

Корреляция нужна, чтобы отличать одиночные симптомы от связанной картины. В проекте есть два уровня корреляции: статические combo findings внутри heuristics и итоговая static/dynamic correlation в assessment.

Combo finding	Почему это усиливает вывод
`placement_exec_nonexec_handler_combo`	unwind metadata в executable section совпадает с handler target в non-exec/unknown section. Два разных placement-слоя подтверждают друг друга.
`placement_integrity_combo`	Странное размещение metadata совпадает с UNWIND_INFO / UnwindInfoRVA integrity problem. Placement подкрепляется повреждением структуры.
`reuse_dominance_combo`	Heavy UnwindInfo reuse совпадает с dominant handler target. Это похоже на centralized dispatcher-style metadata reuse.
`writable_metadata_handler_combo`	Writable unwind metadata совпадает с writable handler targets. Изменяемой выглядит не одна деталь, а вся линия metadata -> handler.

Cross-layer correlation в итоговом assessment включается консервативно: либо есть handler-related static evidence и runtime handler/VEH/VCH/UEF hits, либо есть static placement anomalies и runtime dynamic function-table activity. Это не per-function proof graph, а осторожный признак того, что статическая картина получила динамическое подтверждение.

8. Evidence level и final verdict

В проекте есть два близких, но разных слоя решения. Static evidence level описывает силу статических evidence. Final verdict описывает итоговую категорию отчета с учетом score, runtime-сигналов и correlation.

Static evidence level	Правило
`strong`	strong_count >= 2 или score >= 70
`medium`	strong_count >= 1 или medium_count >= 2 или score >= 30
`weak`	есть finding или score > 0, но до medium/strong не дотянуло
`none`	нет findings и score == 0

Условие	Final verdict
score >= 70	`high_suspicion_static или high_suspicion_correlated`
score >= 40	`moderate_suspicion_static или moderate_suspicion_correlated`
score >= 15	`low_suspicion_static или low_suspicion_correlated`
score < 15 + complete runtime signals	`runtime_signals_only`
нет сильной корреляции/сигналов	`no_strong_correlated_evidence`

Final confidence начинается от static confidence. Если dynamic session complete и есть dynamic signals, добавляется +0.10. Если есть static/dynamic correlation, добавляется еще +0.10. Верхняя граница final confidence - 0.98, чтобы не изображать абсолютную уверенность.

9. Почему некоторые признаки важнее других

Самые важные признаки затрагивают базовую проверяемость PE/EH/unwind карты: OOB, overlap, invalid RUNTIME_FUNCTION ranges, parse fail, повреждение Exception Directory. Если эти структуры противоречивы, анализатор не может надежно построить координатную систему и перейти от RIP/RVA к функции и UNWIND_INFO.

Менее тяжелые признаки связаны с контекстом: unusual section, writable placement, reuse, dominance. Они важны, но у них больше легитимных объяснений: нестандартный компилятор, packer, runtime registration, маленькая выборка, leaf-only image, compatibility toolchain. Поэтому они получают меньший вес, требуют порогов или включаются только при подтверждении.

Combo findings получают дополнительные, но не огромные веса. Причина: базовые признаки уже добавили score, а combo должен усилить связанную картину, не создавая двойной учет одной и той же проблемы.

10. Что проект честно не утверждает

Ограничение	Правильное объяснение
Не вероятность malware	Score и suspicion не означают “70% вирус”. Это сумма инженерных evidence weights.
Не универсальная статистика	Пороги являются defaults, а не ROC-calibrated моделью на большой malware/benign базе.
Не замена reverse engineer-у	Отчет показывает, куда смотреть: RVA, секции, findings, mitigations, manual validation steps.
Не полный EDR	Проект анализирует PE/EH/unwind область и часть dynamic validation, а не всю телеметрию системы.

11. Готовая формулировка для защиты

“Веса и пороги в проекте являются не магическими числами и не статистической вероятностью вредоносности, а инженерной матрицей оценки силы структурных EH/unwind-аномалий. Наибольший вес получают признаки, которые ломают базовую PE/function-table/unwind координатную систему: выходы за границы, перекрытия, parse fail и повреждение Exception Directory. Более мягкие признаки, такие как необычное размещение, reuse или dominance, получают меньший вес, потому что могут иметь легитимные объяснения и требуют контекста. Confidence отдельно показывает устойчивость интерпретации, а mitigations защищают от чрезмерного вывода на одиночных или малых выборках. Итоговый verdict строится не по одному признаку, а по score, severity, семействам evidence, корреляции и runtime-сигналам.”

12. Где это находится в проекте

Файл/папка	Что подтверждает
`docs/evidence_matrix.md`	Фиксирует 27 findings, weights, confidence rules, evidence level, final case rules, correlation и manual review policy.
`src/heuristics.c`	Реализует thresholds, score weights, add_finding, mitigations, confidence, combo findings и requires_manual_review.
`src/report.c`	Формирует assessment: final_verdict, final_confidence, case_strength, static/runtime evidence level и HTML/JSON вывод.
`docs/threshold_calibration.md`	Объясняет, что текущие thresholds - engineering defaults, и задает workflow для threshold sweep на corpus.
`tests/unit/* и tests/samples/*`	Проверяют score, confidence, mitigations, combos, report semantics и sample corpus.

Приложение: точные константы из кода

Эта страница связывает объяснение в PDF с фактическими именами констант из src/heuristics.c и правилами итогового assessment из src/report.c. Ее удобно использовать, если комиссия спросит, где именно в коде находятся пороги и веса.

Пороги эвристик

Константа	Значение	Зачем нужна
`THRESHOLD_HIGH_PERCENT`	`0.01`	Больше 1% плохих записей означает не одиночный сбой, а pattern повреждения.
`THRESHOLD_UNWIND_ANOM_HIGH`	`0.05`	Больше 5% unwind anomalies считается широким integrity-паттерном.
`THRESHOLD_UNWIND_REUSE_MEDIUM`	`0.10`	Reuse больше 10% дает moderate candidate, но finding включается только при corroboration.
`THRESHOLD_UNWIND_REUSE_HIGH`	`0.30`	Reuse больше 30% на n > 200 считается сильной концентрацией metadata.
`THRESHOLD_HANDLER_DOMINANCE`	`0.80`	Один handler target доминирует, если покрывает больше 80% handler-записей при handler_count > 50.

Константы confidence

Константа	Значение	Смысл
`CONFIDENCE_BASE`	`0.15`	Базовая осторожная уверенность.
`CONFIDENCE_ZERO_SCORE_BASE`	`0.20`	База для чистого файла без score.
`CONFIDENCE_PER_FAMILY`	`0.11`	Бонус за независимое evidence family.
`CONFIDENCE_PER_STRONG`	`0.08`	Бонус за high/critical finding.
`CONFIDENCE_PER_MEDIUM`	`0.04`	Бонус за medium finding.
`CONFIDENCE_BONUS_TABLE_HARD`	`0.08`	Бонус за hard table-integrity проблему.
`CONFIDENCE_BONUS_UNWIND_HARD`	`0.08`	Бонус за hard unwind-integrity проблему.
`CONFIDENCE_PENALTY_SINGLE_SIGNAL`	`0.12`	Штраф за одиночный non-strong сигнал.
`CONFIDENCE_PENALTY_SMALL_SAMPLE`	`0.06`	Штраф за маленькую выборку .pdata.
`CONFIDENCE_PENALTY_PLACEMENT_ONLY`	`0.10`	Штраф за isolated placement-only картину.
`CONFIDENCE_PENALTY_SINGLE_FAMILY`	`0.08`	Штраф за evidence из одного family без strong finding.
`CONFIDENCE_MIN / MAX`	`0.05 / 0.90`	Границы статической уверенности.

Финальный verdict в src/report.c

Правила итогового verdict отделены от raw findings. При score >= 70 отчет получает high_suspicion_static или high_suspicion_correlated. При score >= 40 - moderate_suspicion_static/correlated. При score >= 15 - low_suspicion_static/correlated. Если score мал, но завершенная динамическая сессия дала сигналы, используется runtime_signals_only. Correlation дополнительно повышает final_confidence, но не превращает score в вероятность.

Дополнение: почему именно такие числа: +25, +47 и +26

Этот блок добавлен как пояснение к матрице весов. Он отвечает на вопрос комиссии: почему числа в score именно такие, кто их назначает и почему нельзя сказать, что +25 математически единственно правильнее +26.

Кто задает эти числа

В текущей версии проекта веса и пороги задает разработчик как экспертную инженерную policy. Это не обученная ML-модель и не статистическая вероятность вредоносности. Числа выбираются по силе технического нарушения, риску ложного срабатывания, массовости проблемы, наличию подтверждающих сигналов и проверке на тестовом корпусе.

> Правильная формулировка: числа экспертно заданы и зафиксированы в evidence matrix, чтобы результат был воспроизводимым; при появлении большого размеченного corpus их можно перекалибровать.

Почему +25, а не +47

Вес нельзя рассматривать отдельно от всей шкалы. В проекте score ограничен диапазоном `0..100`, а итоговые пороги verdict идут ступенями: `15+` - low suspicion, `40+` - moderate suspicion, `70+` - high suspicion. Поэтому максимальный одиночный вес должен быть сильным, но не должен один почти решать дело.

Вариант	Что происходит
`+25`	Один очень сильный finding сразу заметен и требует ручной проверки, но не превращает файл в high verdict без накопления evidence.
`+25 + +25 = 50`	Два сильных независимых признака дают moderate, то есть уже серьезную картину.
`+25 + +25 + +25 = 75`	Три сильных признака дают high: высокий verdict появляется от совокупности evidence.
`+47`	Один признак почти сразу дает половину шкалы, два признака дают 94 и почти насыщают score. Остальные findings начинают мало влиять.

Именно поэтому `+25` выбран как верхний уровень грубой шкалы: это четверть от 100. Он оставляет место для накопления нескольких признаков, correlation rules, confidence и ручной интерпретации. `+47` сделал бы модель слишком резкой: одиночный дефект начал бы доминировать над всем отчетом.

Почему +25, а не +26

Между `+25` и `+26` нет принципиальной технической пропасти. Если бы проект был статистически калиброван на большой размеченной базе, можно было бы спорить за каждый балл. Но текущая модель специально использует грубые уровни, чтобы не создавать ложную точность.

Вес	Смысл группы
`+8`	мягкий или подтверждающий сигнал
`+10`	умеренный сигнал
`+15`	заметный сигнал
`+20`	сильный сигнал
`+25`	очень сильный сигнал, верхняя группа

Если поставить `+26`, модель не станет содержательно лучше, но появится вопрос: почему не `+27` или `+24`? Поэтому используется округленная сетка. Важен не один балл, а принадлежность признака к группе “очень сильный structural finding”.

Что важно защищать: абсолютные числа или порядок

В rule-based системе важнее не абсолютная цифра сама по себе, а относительный порядок и воспроизводимость. OOB, overlap, parse fail и повреждение Exception Directory должны весить больше, чем unusual placement. Placement, reuse и dominance должны требовать контекста. Combo finding должен усиливать картину, но не дублировать весь вес базовых признаков.

> Честный ответ комиссии: “Я не утверждаю, что +25 математически единственно возможно. Я утверждаю, что это осознанный верхний уровень инженерной шкалы 0..100. Он достаточно сильный, чтобы выделить серьезный structural finding, и достаточно умеренный, чтобы высокий verdict требовал накопления нескольких evidence”.

Готовый ответ на защите

> “Цифры задает разработчик как инженерную policy. Они не являются физической величиной или вероятностью. `+25` выбран как верхний уровень дискретной сетки весов и как четверть шкалы `0..100`: один сильный признак требует внимания, два сильных признака дают moderate, три - high. `+47` сделал бы одиночный признак чрезмерно доминирующим, а `+26` создал бы ложную точность без нового смысла. На большом размеченном корпусе эти веса можно перекалибровать, но в текущей работе важны прозрачность, объяснимость и воспроизводимость матрицы”.

Как это связано с тестами

После изменения любой цифры нужно заново прогонять unit tests, regression samples и score sweep. Иначе меняется поведение final verdict: samples могут перескочить из low в moderate или из moderate в high. Поэтому веса в проекте зафиксированы в evidence matrix: не потому что другие числа невозможны, а потому что конкретная версия должна быть стабильной и проверяемой.